Fatec

Faculdade de Tecnologia

Fatec
Registro

Prof. Maylon H. Oliveira

SÃO PAULO
GOVERNO DO ESTADO

FATEC | CPS — Faculdade de Tecnologia

LISTA DE EXERCÍCIOS RESOLVIDOS TRANSFORMAÇÕES LINEARES + PYTHON

Resolução Algébrica, Código NumPy e Interpretação Geométrica das Questões 1 a 12

Disciplina: Álgebra Linear — MAG004 | Curso: Tecnologia em DSM

Fatec Registro

Prof. Maylon H. Oliveira

SÃO PAULO GOVERNO DO ESTADO

PARTE 1 — APLICAÇÃO DIRETA E MATRIZES

Questões 1 a 4: Resolução algébrica, produto matricial e testes de linearidade

Questão 1 — Aplicando uma Transformação Linear

Resolução Algébrica

$T(x, y) = (3x, 2y)$ e $v = (2, -1)$

Passo a Passo da Substituição:

1. Identificamos as componentes do vetor de entrada: x = 2 e y = -1
2. Substituímos diretamente na regra da transformação T(x,y) = (3x, 2y):

$T(2, -1) = (3(2), 2(-1)) = (6, -2)$

Questão 1 — Implementação em Python

```
import numpy as np

# Definindo o vetor de entrada v = (2, -1)
v = np.array([2, -1])

# Aplicando a transformação T(x, y) = (3x, 2y)
resultado = np.array([
    3 * v[0],
    2 * v[1]
])

print("Resultado T(v):", resultado)
# Saída esperada: [ 6 -2]
```

Explicação do Código

- Array NumPy:
Utilizamos np.array() para representar o vetor tridimensional/bidimensional.

- Indexação por Índice:
v[0] acessa x (2) e v[1] acessa y (-1).

- Vetor Resultante:
O resultado [6, -2] confirma com precisão a resolução matemática.

Questão 2 — Transformação Representada por Matriz

Resolução Algébrica por Produto Matricial

Cálculo Linha por Coluna:

- Primeira linha de A × vetor v: 2·(4) + 1·(2) = 8 + 2 = 10
- Segunda linha de A × vetor v: 0·(4) + 3·(2) = 0 + 6 = 6

$$T(v) = Av = [2(4) + 1(2),\ 0(4) + 3(2)]^T = [10, 6]^T$$

$$A = \begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix}$$

$$v = \begin{bmatrix} 4 \\ 2 \end{bmatrix}$$

Questão 2 — Implementação em Python

```
import numpy as np

# Definindo a matriz associada A (2x2)
A = np.array([
    [2, 1],
    [0, 3]
])

# Definindo o vetor coluna v (2x1)
v = np.array([4, 2])

# Multiplicação matricial com o operador @
resultado = A @ v

print("Resultado T(v) = A @ v:", resultado)
# Saída esperada: [10  6]
```

O Operador @ no NumPy

- Operador @:
No Python 3.5+, o operador @ realiza a multiplicação matricial nativa do NumPy (equivalente a np.dot(A, v)).

- Eficiência Computacional:
Ideal para aplicar transformações lineares em grandes volumes de dados ou coordenadas 2D/3D.

Questão 3 — Verificando se uma Transformação é Linear

Demonstração pela Existência da Matriz Associada

$T(x, y) = (2x + y,\ x - 3y)$

Argumento Algébrico:

Podemos expressar as componentes de T(x,y) como uma combinação linear pura das variáveis x e y:

$$T(x,y) = \begin{bmatrix} 2 & 1 \\ 1 & -3 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Questão 3 — Validação Numérica da Linearidade

```
import numpy as np

A = np.array([[2, 1], [1, -3]])

u = np.array([2, 1])
v = np.array([-1, 3])
alpha, beta = 4, -2

# Testando T(alpha*u + beta*v)
lado_esquerdo = A @ (alpha * u + beta * v)

# Testando alpha*T(u) + beta*T(v)
lado_direito = alpha * (A @ u) + beta * (A @ v)

print("Lado Esquerdo:", lado_esquerdo)
print("Lado Direito: ", lado_direito)
print("É Linear?", np.array_equal(lado_esquerdo, lado_direito))
# Saída esperada: True
```

Propriedade Testada

$T(\alpha u + \beta v) = \alpha T(u) + \beta T(v)$

- Teste de Combinação: Ao verificar que T(αu + βv) = αT(u) + βT(v), confirmamos numericamente a preservação conjunta da soma e do escalar.

Questão 4 — Exemplo de Transformação Não Linear

Aplicação do Teste Rápido do Vetor Zero

$T(x, y) = (x + 2, y)$

Demonstração por Contradição:

Toda transformação linear necessita satisfazer obrigatoriamente a condição T(0,0) = (0,0).

Avaliando a função no vetor nulo: T(0,0) = (0+2, 0) = (2,0).

$T(0, 0) = (0 + 2, 0) = (2, 0) \neq (0, 0) \rightarrow$ Não Linear

Questão 4 — Implementação em Python

```
import numpy as np

# Definindo a função T(x, y) = (x + 2, y)
def T(v):
    return np.array([
        v[0] + 2,
        v[1]
    ])

zero = np.array([0, 0])
resultado_zero = T(zero)

print("T(0, 0) =", resultado_zero)
print("É igual a (0,0)?", np.array_equal(resultado_zero, zero))
# Saída esperada: [2 0], False -> Não Linear!
```

Conclusão do Teste

• Translação Rigida:
A presença da constante +2 representa uma translação no espaço. Translações não são transformações lineares pois deslocam a origem.

PARTE 2 — GEOMETRIA, POSIÇÃO E ESCALA

Questões 5 a 8: Escala, reflexão no eixo Y, transformação de polígonos e matrizes 2x2

Questão 5 — Escala de um Ponto

Transformação de Escala Anisotrópica

$T(x, y) = (3x, 2y), \quad P = (3, 4)$

Aplicação da Escala:

- Multiplicando a coordenada x por 3 e y por 2:

$T(3, 4) = (3(3), 2(4)) = (9, 8)$

Questão 5 — Implementação em Python

```
import numpy as np

# Matriz de Escala Anisotrópica (Sx = 3, Sy = 2)
A = np.array([
    [3, 0],
    [0, 2]
])

P = np.array([3, 4])
novo_P = A @ P

print("Ponto transformado P':", novo_P)
# Saída esperada: [9 8]
```

Matrizes Diagonais

- Fatores de Escala:
Em matrizes diagonais, os elementos a11 e a22 representam os fatores de ampliação/redução dos eixos X e Y.

Questão 6 — Reflexão em Relação ao Eixo Y

Inversão de Sinal na Coordenada X

$T(x, y) = (-x, y), \quad P = (5, -2)$

Resolução Algébrica:

- Substituindo x = 5 e y = -2 em T(x,y) = (-x, y):

$T(5, -2) = (-5, -2)$ (Reflexão no Eixo Y)

Questão 6 — Implementação em Python

```
import numpy as np

# Matriz de Reflexão em relação ao Eixo Y
A = np.array([
    [-1,  0],
    [ 0,  1]
])

P = np.array([5, -2])
resultado = A @ P

print("P' após reflexão:", resultado)
# Saída esperada: [-5 -2]
```

Efeito Geométrico

- Espelhamento Lateral:

O sinal negativo em a11 = -1 inverte o sentido do eixo horizontal, criando um efeito de espelho em relação ao eixo vertical (Y).

Questão 7 — Transformação de Vários Pontos

Transformação Uniforme de um Triângulo (Escala 2x)

$P_1 = (1, 2), \quad P_2 = (4, 1), \quad P_3 = (2, 5)$

Aplicações em Cada Vértice:

- A·P1 = [2(1), 2(2)]^T = (2,4) | A·P2 = [2(4), 2(1)]^T = (8,2) | A·P3 = [2(2), 2(5)]^T = (4,10)

$P'_1 = (2, 4), \quad P'_2 = (8, 2), \quad P'_3 = (4, 10)$

Questão 7 — Processamento Vetorizado em Python

```
import numpy as np

A = np.array([[2, 0], [0, 2]])

# Matriz de pontos (3 pontos x 2 coordenadas)
pontos = np.array([
    [1, 2],
    [4, 1],
    [2, 5]
])

# Multiplicação vetorizada: pontos @ A.T
resultado = pontos @ A.T

print("Novas Coordenadas:
", resultado)
# Saída esperada:
# [[ 2  4]
#  [ 8  2]
#  [ 4 10]]
```

Multiplicação Vetorizada

• Alta Performance:
Em vez de iterar com loops for, multiplicamos a matriz de pontos inteira por A.T, transformando milhares de vértices instantaneamente.

Questão 7 — Desenhando com matploit

```
import matplotlib.pyplot as plt

# Pontos
P1 = (1, 2)
P2 = (4, 1)
P3 = (2, 5)

# Coordenadas para desenhar o triângulo
x = [P1[0], P2[0], P3[0], P1[0]]
y = [P1[1], P2[1], P3[1], P1[1]]

# Plotando o triângulo
plt.plot(x, y, marker='o')

# Identificando os pontos
plt.text(P1[0], P1[1], ' P1 (1,2)')
plt.text(P2[0], P2[1], ' P2 (4,1)')
plt.text(P3[0], P3[1], ' P3 (2,5)')
```

```
# Eixos
plt.axhline(0)
plt.axvline(0)

# Grade
plt.grid(True)

# Escala igual nos dois eixos
plt.axis('equal')

# Títulos
plt.xlabel('x')
plt.ylabel('y')
plt.title('Triângulo no Plano Cartesiano')

plt.show()
```

Questão 7 — Desenhando vetores com matploit

```python
import matplotlib.pyplot as plt

# Vetor u
plt.quiver(
    0, 0,       # início
    2, 1,       # componentes
    angles='xy',
    scale_units='xy',
    scale=1
)

# Vetor v
plt.quiver(
    0, 0,
    -1, 3,
    angles='xy',
    scale_units='xy',
    scale=1
)

# Eixos
plt.axhline(0)
plt.axvline(0)

# Grade
plt.grid(True)

# Escala igual
plt.axis('equal')

# Limites
plt.xlim(-3, 4)
plt.ylim(-1, 5)

plt.xlabel('x')
plt.ylabel('y')
plt.title('Vetores u e v')

plt.show()
```

Questão 8 — Descobrindo o Ponto Transformado

Aplicação de Matriz de Cisalhamento/Combinação

Cálculo do Ponto Transformado:

- Primeira Coordenada: 1·(2) + 2·(3) = 2 + 6 = 8
- Segunda Coordenada: 3·(2) + 1·(3) = 6 + 3 = 9

$$T(P) = [1(2) + 2(3),\ 3(2) + 1(3)]^T = (8, 9)$$

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 1 \end{bmatrix}$$

$$P = (2, 3)$$

Questão 8 — Implementação em Python

```
import numpy as np

# Matriz A e Ponto P
A = np.array([
    [1, 2],
    [3, 1]
])

P = np.array([2, 3])

# Multiplicação matricial
p_transformado = A @ P

print("Ponto Transformado T(P):", p_transformado)
# Saída esperada: [8 9]
```

Resumo da Operação

- Mapeamento de Pontos:

O ponto original (2,3) é mapeado diretamente no novo ponto (8,9) no plano cartesiano por meio do operador @.

PARTE 3 — PROPRIEDADES DA SOMA E APLICAÇÕES

Questões 9 e 10: Preservação da adição e movimento de personagens em jogos 2D

Fatec Registro

Prof. Maylon H. Oliveira

SÃO PAULO GOVERNO DO ESTADO

Questão 9 — Verificando a Preservação da Adição

Verificação da Propriedade T(u + v) = T(u) + T(v)

$T(x, y) = (x - y, 2x + y), \quad u = (3, 1), \quad v = (2, -2)$

Objetivo da Questão:

- Devemos calcular o lado esquerdo T(u+v) e comparar com o lado direito T(u) + T(v).
- Primeiro passo: Somar os vetores de entrada u + v = (3+2, 1+(-2)) = (5, -1).

Questão 9 — Resolução Passo a Passo

Comparação dos Resultados

1. Calculando T(u + v) = T(5, -1):

$$T(u+v) = T(5, -1) = (6, 9)$$

2. Calculando T(u) + T(v) separadamente:

- **T(3,1) = (3-1, 2(3)+1) = (2,7)**
- **T(2,-2) = (2-(-2), 2(2)+(-2)) = (4,2)**

$$T(u) + T(v) = (2, 7) + (4, 2) = (6, 9) \rightarrow T(u+v) = T(u) + T(v)$$

Questão 9 — Implementação em Python

```
import numpy as np

A = np.array([[1, -1], [2, 1]])
u = np.array([3, 1])
v = np.array([2, -2])

# Lado Esquerdo: T(u + v)
esquerda = A @ (u + v)

# Lado Direito: T(u) + T(v)
direita = (A @ u) + (A @ v)

print("T(u + v):  ", esquerda)
print("T(u) + T(v):", direita)
print("São iguais?", np.array_equal(esquerda,
direita))
# Saída: [6 9], [6 9], True
```

Confirmação Numérica

- Preservação da Soma:

O Python confirma que a ordem das operações (somar antes ou transformar antes) produz exatamente o mesmo vetor de saída [6, 9].

Questão 10 — Aplicação: Movimento de um Objeto 2D

Em um jogo 2D, a posição de um personagem é:

$$P = (7, 3)$$

O vetor de movimento é:

$$d = (-2, 4)$$

Determine a nova posição:

$$P' = P + d$$

Cálculo da Nova Posição:

- Somando componente a componente:

P' = (7 + (-2), 3 + 4)

$$P' = (7, 3) + (-2, 4) = (5, 7)$$

Questão 10 — Implementação em Python

```
import numpy as np

# Posição inicial do personagem em R²
posicao = np.array([7, 3])

# Vetor de movimento (deslocamento)
movimento = np.array([-2, 4])

# Atualização de posição P' = P + d
nova_posicao = posicao + movimento

print("Nova Posição P':", nova_posicao)
# Saída esperada: [5 7]
```

Vetores na Computação

• Conceito Prático:
Vetores não são apenas abstrações puras: em motores de jogos (Unity, Unreal, Godot), representam posições, velocidades e forças de maneira direta.

PARTE 4 — DESAFIOS FINAIS INTEGRADORES

Questões 11 e 12: Transformação de Imagens e Desafio Completo de Matemática + Programação

Questão 11 — Desafio: Transformação de uma Imagem

Transformação de um Polígono de 4 Vértices (Escala 2x no Eixo X)

$P_1 = (1, 1), \quad P_2 = (3, 1), \quad P_3 = (3, 3), \quad P_4 = (1, 3)$

Operação de Escala Horizontal:

• Cada coordenada x é multiplicada por 2, enquanto a coordenada y permanece inalterada (multiplicada por 1).

$P'_1 = (2, 1), \quad P'_2 = (6, 1), \quad P'_3 = (6, 3), \quad P'_4 = (2, 3)$

$$A = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}$$

Questão 11 — Processamento da Imagem em Python

```
import numpy as np

A = np.array([[2, 0], [0, 1]])

# Imagem representada por 4 pontos
imagem = np.array([
    [1, 1],
    [3, 1],
    [3, 3],
    [1, 3]
])

# Processamento vetorizado
nova_imagem = imagem @ A.T

print("Vértices Transformados:
", nova_imagem)
# Saída esperada:
# [[2 1]  [6 1]  [6 3]  [2 3]]
```

Resultado do Código

• Matriz de Vértices:
A matriz 4x2 armazena toda a figura geométrica. A multiplicação por A.T aplica a escala a todos os vértices simultaneamente.

Questão 11 — Análise e Interpretação Geométrica

O Que Aconteceu com a Figura?

- Figura Original:

Um quadrado de lado 2, com vértices em (1,1), (3,1), (3,3) e (1,3).

- Figura Transformada:

Um retângulo de largura 4 e altura 2, com vértices em (2,1), (6,1), (6,3) e (2,3).

Conclusão Geométrica: A figura foi ALONGADA HORIZONTALMENTE por um fator de 2!

Questão 12 — Desafio Final: Matemática + Programação

Enunciado Completo do Desafio

$$T(x, y) = \begin{pmatrix} 2x + y \\ x + 2y \end{pmatrix}$$

$$u = (1, 3)$$

$$v = (4, -1)$$

Tarefas a Resolver:

a) Calcule T(u)
b) Calcule T(v)
c) Calcule T(u + v)
d) Verifique se T(u + v) = T(u) + T(v)
e) Implemente tudo em Python.

Questão 12 — Resolução dos Itens (a), (b) e (c)

Cálculo das Imagens dos Vetores

a) Para u = (1,3): T(1,3) = (2(1)+3, 1+2(3)) = (5, 7)

b) Para v = (4,-1): T(4,-1) = (2(4)+(-1), 4+2(-1)) = (7, 2)

c) Para u + v = (5,2): T(5,2) = (2(5)+2, 5+2(2)) = (12, 9)

$$T(u) = (5, 7), \quad T(v) = (7, 2), \quad T(u + v) = (12, 9)$$

Questão 12 — Resolução do Item (d)

Prova de Preservação da Adição para u e v

Comparando os Lados da Igualdade:

- Lado Esquerdo: T(u + v) = (12, 9)
- Lado Direito: T(u) + T(v) = (5, 7) + (7, 2) = (12, 9)

$$T(u+v) = T(u) + T(v) \rightarrow (12, 9) = (5, 7) + (7, 2)$$

Questão 12 — Implementação Completa em Python

```
import numpy as np

A = np.array([[2, 1], [1, 2]])
u = np.array([1, 3])
v = np.array([4, -1])

Tu = A @ u
Tv = A @ v
Tuv = A @ (u + v)
soma = Tu + Tv

print("a) T(u) =", Tu)      # [5 7]
print("b) T(v) =", Tv)      # [7 2]
print("c) T(u+v) =", Tuv)   # [12 9]
print("d) T(u)+T(v) =", soma) # [12 9]
print("É linear?", np.array_equal(Tuv, soma)) # True
```

Resumo da Validação

• Fechamento do Desafio:
O script Python resolve todos os itens de a até e em poucas linhas de código, comprovando a eficácia das matrizes para representar transformações.